



# 隐盾安全智能体

面向涉密办公场景的本地化隐私防护桌面智能体

YINDUN SECURITY AGENT — A Privacy-Gateway Desktop Agent for Confidential Office Work

## 摘要

大模型进入办公场景面临一个根本矛盾：待处理数据越敏感，越需要模型的能力，却也越不能让其跨越本地边界。本工作报告提出并实现隐盾安全智能体，其核心是一个内存级隐私脱敏网关——将敏感文本在进入模型前形式化地映射为占位符文本与一份加密映射表，使模型在信息论意义上无法接触真实敏感值，同时保持任务可用性。系统配套本地沙箱、命令白名单、高危熔断审批与 HMAC 哈希链审计，构成覆盖“输入—推理—执行—落盘—审计”的纵深防御。为量化隐私防护效果，我们构建了可复现的攻防评测：在 13 篇含 41 个敏感实体实例的合成办公语料上，黑盒出网脱敏召回率 **100%**、明文泄露 **0**、业务数字误报率 **0%**、还原正确率 **100%**；还原劫持攻击 500 轮成功 **0**；提示注入下模型可见明文 PII **0**；白盒持久化与审计产物明文命中 **0**。结果表明该机制在既定威胁模型下能有效阻断敏感数据的出网与落盘泄露。

系统

隐盾安全智能体

版本

V3.3 (2026-09)

技术栈

Python · LangChain · PySide6 · Chroma

# 目录

1 研究背景与动机

2 关键挑战

3 系统概览与架构

4 隐私保护机制

5 攻击实验与结果

6 纵深防御与审计

7 安全工程闭环

8 测试与质量保障

9 结论与展望

## 1 研究背景与动机

大模型办公助手正在快速普及，但企业与机构在涉密文档（合同、薪酬、病历、客户资料）处理上面临一个根本矛盾：**数据越敏感，越需要 AI 的能力，却也越不能把数据交出去**。无论是调用云端 API，还是把对话写入本地日志、向量库与审计文件，每一个跨越本地边界的环节都可能成为泄露面。

现有的通用做法存在结构性缺陷。"提示用户不要粘贴敏感信息"依赖人的自觉而不可靠；"整体关闭联网"则牺牲了云端强模型的能力。二者都没有把隐私保护变成系统本身的属性。我们的出发点是：**把隐私保护从"使用约定"下沉为"架构约束"**——让大模型在物理上"看不见"敏感数据，却依然能把活干完。这既保留了可用性，又从机制上消除了泄露面。

围绕这一目标，我们把办公智能体的泄露面归纳为四类，并逐一设计防线：

### 出网面

敏感明文随 prompt 发往云端，离开本地即失控  
→ 脱敏网关。

### 落盘面

会话记录、向量库、摘要把真实隐私写进磁盘 →  
加密映射 + 入库即脱敏。

### 留痕面

审计日志本为追溯，却可能反成二次泄露源 → 审计只存脱敏预览。

### 执行面

智能体具备写盘/删除/执行能力，被诱导即造成物理破坏 → 沙箱 + 白名单 + 熔断审批。

## 2 关键挑战

把上述目标做成一个可用的系统，需跨越四道核心挑战：

**其一，可用性与不可见性的张力。**若把敏感值简单删除，模型会因上下文断裂而无法作答。必须在"抹去真实值"与"保留语义结构"之间取得平衡——这正是我们采用占位符化而非删除的设计动机：占位符保留实体类型与位置信息，使模型仍能理解"这里有一个手机号"并据此组织回答。

**其二，识别的召回与精确难以两全。**中文办公文本里，手机号、身份证、银行卡、人名、金额形态多变（全角、分段、国际前缀、单位混排），既要高召回不漏，又不能误伤订单号、版本号、日期等业务数字。我们以"正则粗筛 + 程序化精验"两段式应对。

**其三，防护必须贯穿全链路，而非只在入口。**脱敏若只做在用户输入，落盘的会话、审计、向量库、以及模型输出的还原环节仍会泄露；任一环节旁路，整体即失效。这要求把同一套脱敏管线强制施加到所有数据出入口。

**其四，可证明性。**"隐私有效"不能靠宣称，需要在明确的威胁模型下、以可复现的攻击实验来度量。这驱动我们设计了第 5 节的攻防评测。

## 3 系统概览与架构

系统采用「GUI 层 — 推理引擎层 — 安全内核层」三层解耦。安全内核由隐私引擎、策略引擎、密钥管理、审计日志四个独立模块组成，可脱离 GUI 单独测试与评测。数据在系统内的旅程为：

用户输入  $x$  → 脱敏  $\hat{x}$  → ReAct 推理 / 工具调用 → 结果脱敏 → 还原  $D(\hat{y})$  → 呈现用户

其中模型侧全程只接触  $\hat{x}$  与占位符化的工具返回；每一环节的事件均同步写入哈希链审计。算力支持本地 Ollama（数据不出网）与 OpenAI 兼容云端（出网内容已脱敏）双模式。

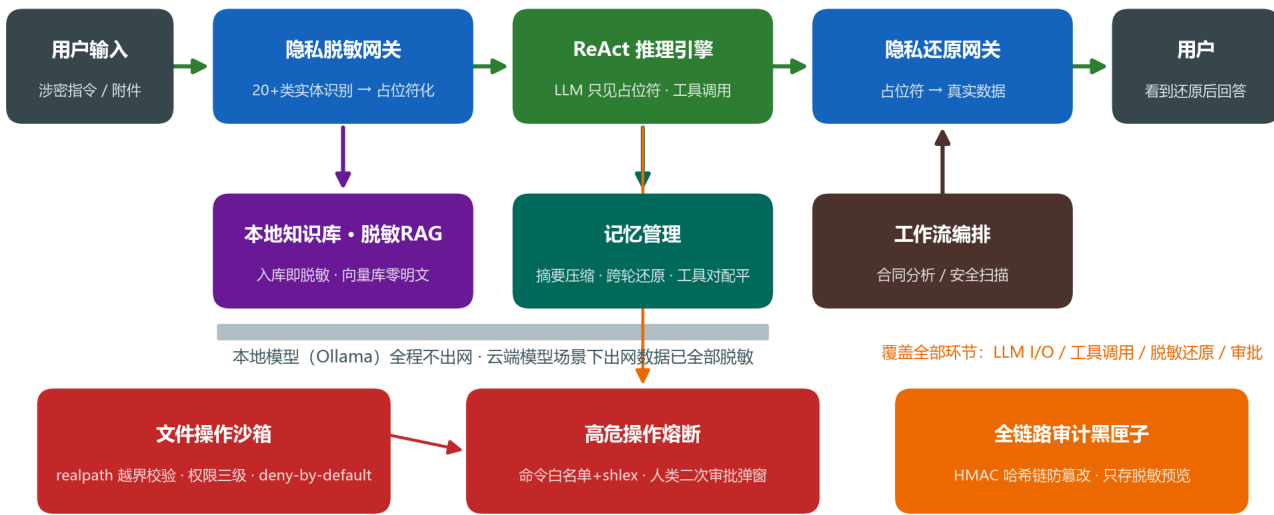


图 1 核心数据流与安全防线：脱敏网关位于输入/输出两侧，安全内核覆盖执行与审计全程

## 4 隐私保护机制

### 4.1 脱敏算法的形式化定义

设文本全集为  $\mathcal{X}$ ，敏感实体识别器  $\mathcal{E} : \mathcal{X} \rightarrow 2^{\mathcal{X}}$  输出文本中的敏感实体集合。隐私网关定义为一个随机化映射：

$$A_k : \mathcal{X} \longrightarrow \hat{\mathcal{X}} \times \mathcal{M}, \quad A_k(x) = (\hat{x}, M), \quad M = \{p_i \mapsto \text{Enc}_k(e_i)\}_{e_i \in \mathcal{E}(x)}$$

其中  $\hat{x}$  是将每个实体  $e_i$  替换为占位符  $p_i$  后的文本（送入模型）， $M$  为映射表，真实值以密钥  $k$  下的 Fernet 密文存储。占位符形如  $p_i = [\text{Type}_{i\_i\_}r_i]$ ， $r_i \leftarrow \{a-z, 0-9\}^4$  为随机 nonce。还原算子  $D_k$  将输出中的占位符解回明文：

$$D_k(\hat{y}, M) = \hat{y} \prod_{p_i \in M} [p_i \leftarrow \text{Dec}_k(M[p_i])]$$

系统在既定威胁模型下主张四条性质：

- **机密性 (Confidentiality)**：对任意  $e \in \mathcal{E}(x)$ ， $\Pr[e \subseteq \hat{x}] = 0$ ——模型侧无明文（实验 5.2 测得 41/41 满足）。
- **可逆性 (Recoverability)**： $D_k(A_k(x)) = x$ （实验 5.2 还原正确率 100%）。
- **还原不可伪造 (Unforgeability)**：攻击者在不掌握  $M$  时构造命中他人映射的字面量，成功概率受 nonce 空间约束  $\Pr[\text{collide}] \leq 36^{-4} \approx 5.9 \times 10^{-7}$ （实验 5.4 劫持 0/500）。

- 持久化机密 (Persistence-Confidentiality): 无密钥  $k$  时, 磁盘产物  $\hat{x}, M, \text{Audit}, \text{Vector}$  均不可恢复明文 (实验 5.5–5.8 命中 0)。

**性质边界 (诚实声明):** 本机制是确定性脱敏而非差分隐私, 不主张对"未被规则覆盖的实体类型"或"可由背景知识重识别的属性组合"提供保证; 人名识别依赖上下文触发词, 无触发词的连续人名列举可能漏检; 算法精验 (Luhn / 校验位) 会主动放过非法格式数字, 属"宁漏不误伤"的既定取舍。

## 4.2 识别与精验流水线

识别采用两段式: 正则  $R_e$  负责召回, 程序化校验器  $V_e$  负责精确, 实体  $e$  被判定为敏感当且仅当  $e \in R_e \wedge V_e(e)$ 。校验器针对不同实体类型实例化: 手机号归一化后须恰为 11 位、银行卡须过 Luhn、身份证须过 GB11643 校验位 (可选)、15 位证须命中地区码或触发词、人名须"姓氏表  $\cap$  角色触发词窗口"。覆盖 20+ 类实体并按 GB/T 35273 思想做三级数据分级。

## 4.3 密钥体系

映射表真实值经 Fernet (AES-128-CBC + HMAC-SHA256) 加密:  $\text{Enc}_k(\cdot)$ 。主密钥  $k$  由密钥管理器统一托管, Windows 下经 DPAPI 按当前用户账户包裹:  $k_{\text{stored}} = \text{DPAPI}_{\text{user}}(k)$ , 磁盘仅存  $k_{\text{stored}}$ , 密钥文件移出项目目录并收紧 ACL。由此将"能否解密"归约为"是否为该 Windows 登录用户", 构成持久化机密性的信任根。

## 4.4 威胁模型与攻击博弈

我们按攻击者能力定义四类对手, 并以博弈方式刻画安全性。记脱敏-还原系统为  $\Pi = (A_k, D_k)$ , 敏感实体集  $\mathcal{E}(x)$ 。

| 对手    | 能力                                        | 对应防线                             |
|-------|-------------------------------------------|----------------------------------|
| 黑盒    | 仅观察出网文本 $\hat{x}$ 与模型输出 (被攻破的模型服务方、链路窃听器) | 机密性: $\hat{x}$ 无明文, 注入亦无法泄露未掌握数据 |
| 黑盒·主动 | 可向待处理文档植入内容 (伪造占位符、注入越权指令)                | 还原不可伪造 (nonce); 指令不改变"模型看不到明文"   |
| 白盒    | 读取本地磁盘持久化文件 (会话/审计/映射/向量库), 但不在活动用户会话内    | 持久化机密: 真实值密文存储, 无 $k$ 不可解        |
| 白盒·持钥 | 劫持当前 Windows 用户会话、可调用 DPAPI 取回 $k$        | 超出防护目标: 等价"用户本人", 是安全与可恢复性的既定边界  |

以出网泄露博弈为例:  $\text{Exp}_{\Pi}^{\text{leak}}(\mathcal{D})$  中, 对手从语料  $\mathcal{D}$  取  $x$ , 观察  $\hat{x} = A_k(x)$ , 输出一个猜测值  $e^*$ ; 若  $e^* \in \mathcal{E}(x)$  则对手胜。机密性等价于  $\Pr[\text{Exp}^{\text{leak}} \Rightarrow 1] = 0$ 。还原劫持博弈  $\text{Exp}^{\text{hijack}}$  中, 对手植入字面量占位符  $p^*$  诱导  $D_k$  将他人明文写入其可控输出, 成功概率受 nonce 空间界定。

# 5 攻击实验与结果

## 5.1 评测方法

我们实现可复现的攻防评测脚本 `privacy_eval.py`。语料为 13 篇合成办公文本，注入 **41 个敏感实体实例**（覆盖国际前缀、全角、分段卡号、PEM/JWT、IPv6/MAC 等边界格式），并含 17 个不应被误伤的业务数字负样本（版本号、日期、订单号、数量）。评测以第 4.4 节的博弈定义度量各性质，全部数据为虚构。

## 5.2 总体结果

| #     | 实验（威胁）                                | 指标                    | 结果           |
|-------|---------------------------------------|-----------------------|--------------|
| 5.2.1 | 黑盒·出网泄露 <b>Exp<sup>leak</sup></b>     | 脱敏召回率 / 明文泄露          | 100% / 0 例   |
| 5.2.2 | 黑盒·误伤                                 | 业务数字误报率               | 0% (0/17)    |
| 5.2.3 | 可用性·往返                                | $D_k(A_k(x)) = x$ 正确率 | 100% (13/13) |
| 5.2.4 | 黑盒主动·还原劫持 <b>Exp<sup>hijack</sup></b> | 植入字面量劫持成功率            | 0 / 500 轮    |
| 5.2.5 | 黑盒主动·提示注入                             | 模型可见上下文真实 PII 数       | 0            |
| 5.2.6 | 白盒·持久化扫描                              | 落盘文件明文 PII 命中         | 0            |
| 5.2.7 | 白盒·审计预览                               | 审计掩码后明文 PII 命中        | 0            |
| 5.2.8 | 白盒·无密钥还原                              | strict 模式是否泄露明文       | 否（保留占位符）     |

表 1 隐私防护攻防评测结果 (privacy\_eval.py, 2026-09-13 复现)

## 5.3 按类别的专项召回测试

为细粒度验证各类识别器的有效性，我们另针对 11 类实体分别构造专项样本（每类 1–4 例、共 25 例，含边界格式），与 5.1 的 41 实例语料相互独立。结果如下：

| 类别     | 召回  | 类别   | 召回  | 类别    | 召回    |
|--------|-----|------|-----|-------|-------|
| 手机号    | 3/3 | 银行卡  | 2/2 | 金额    | 3/3   |
| 身份证    | 2/2 | 邮箱   | 2/2 | IP 地址 | 2/2   |
| API 密钥 | 2/2 | 人名   | 4/4 | 地址    | 2/2   |
| JWT    | 1/1 | 口令字段 | 2/2 | 合计    | 25/25 |

表 2 11 类实体专项样本召回 (25 例，独立于表 1 的 41 实例语料)

**关键结论：**5.2.1 与 5.2.5 合起来构成核心保证——出网文本零明文，因此模型在信息论上无法被诱导泄露它从未接触的数据；5.2.4 证明还原环节不可被植入劫持（碰撞概率  $\leq 36^{-4}$ ）；5.2.6–5.2.8 证明即使攻击者拿走本地磁盘文件、在无密钥条件下也无法恢复明文。

局限：评测语料为合成样例，真实语料上的召回会因未覆盖的实体形态而下降；人名连续列举、依赖背景知识的属性组合重识别不在本机制保证范围内。评测脚本与结果 JSON 随仓库提供，可一键复现。

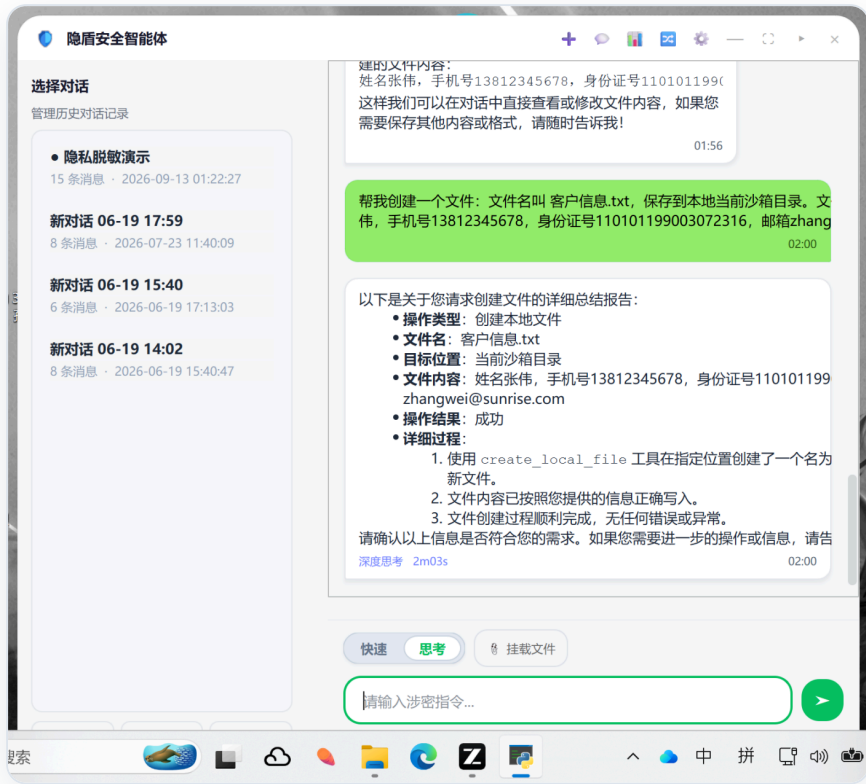


图 2 涉密指令实测：模型调用工具、审批放行、文件创建、回复还原的完整闭环

## 6 纵深防御与审计

隐私之外，智能体的物理副作用同样需要约束。系统对每个具备副作用的工具施加四层防御，并以哈希链审计全程留痕。

### 路径沙箱

对目标路径取 `realpath` 后以 `commonpath` 判定是否落在沙箱根内，先化解符号链接/junction；文件名强制 `basename` 化，杜绝 `../` 穿越；越界读写一律拒绝。

### 命令白名单

命令名须命中 `{python, pip, git, echo}`，经 `shlex` 解析为 `argv` 后以 `shell=False` 执行，参数级拦截 `shell` 操作符、路径穿越与绝对系统路径，从机制上杜绝命令注入。



## 熔断审批

策略引擎（deny-by-default）判定的高危操作挂起后台线程，弹窗展示申请工具与目标路径，人工授权才放行；超时自动驳回并关闭残留弹窗，杜绝"死弹窗"误导。

## 哈希链审计

每条日志哈希  $h_i = \text{HMAC}_\kappa(h_{i-1} \parallel \text{entry}_i)$ ，篡改即校验失败；审计自身落盘前统一复用脱敏管线掩码，只存脱敏预览（见 5.2.7）。

另有模式级防线：快速模式禁用一切写操作工具（只读安全），思考模式支持完整工具调用与可调推理深度。本地知识库采用"入库即脱敏"管线，向量库中只存占位符文本。



图 3 高危操作熔断审批弹窗（实测）

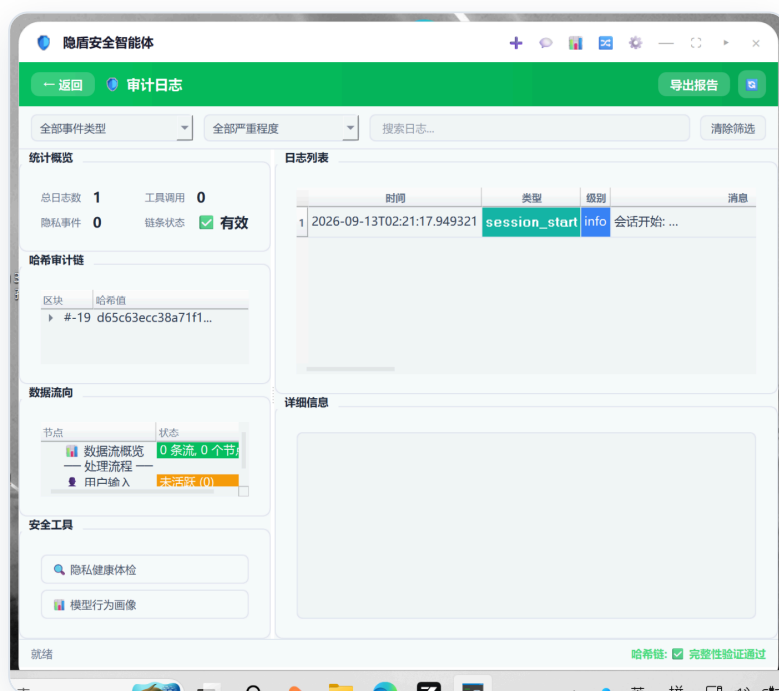


图 4 审计黑匣子面板：统计概览 / 哈希链状态 / 事件筛选 / 报告导出

## 7 安全工程闭环

我们以"红队—修复—回归"的方式持续加固：先对系统做对抗性安全测试（攻击用例动态验证 + 通读源码审查），输出分级问题清单，逐项修复并将对抗用例固化为回归测试。下表列出与安全内核强相关的核心问题及其修复与对应提交：



| 核心问题                                          | 危害                       | 修复                                        | 提交                      |
|-----------------------------------------------|--------------------------|-------------------------------------------|-------------------------|
| 命令执行为 shell=True + 子串黑名单，可被 cmd/powershell 绕过 | 任意命令执行，沙箱形同虚设            | 白名单 + shlex + shell=False + 参数级纵深拦截       | <a href="#">052f054</a> |
| "沙箱"默认放行任意绝对路径，策略 default=approve             | 读桌面/写系统目录均被放行            | realpath+commonpath 越界判定，deny-by-default  | <a href="#">052f054</a> |
| 审批弹窗共用单一 Event，多工具待审时点一个=放行全部                 | 审批旁路，高危操作绕过人工确认          | 逐请求审批 + 超时自动驳回 + 关闭残留弹窗                   | <a href="#">6577cdc</a> |
| workflow 链路绕过隐私网关，合同原文直发云端                    | 聊天侧脱敏在 workflow 侧失效，明文出网 | 所有 LLM 调用统一走 anonymize→invoke→deanonymize | <a href="#">052f054</a> |
| 审计日志在还原后记录，真实 PII 落盘                          | 审计系统反成泄露源                | 落盘前统一掩码（复用完整脱敏管线）                         | <a href="#">3876638</a> |
| 占位符格式固定可预测，可被植入字面量劫持还原                        | 恶意文档诱导写出他人明文             | 随机 nonce + strict 还原不回退明文                 | <a href="#">3876638</a> |
| KeyManager 哈希链不覆盖 cipher_key                  | 可静默替换密钥而完整性校验不报警         | 完整性字段纳入密钥材料                               | <a href="#">052f054</a> |
| 映射表明文 dict 落盘                                 | 拿到文件即还原全部真实隐私            | Fernet 加密存储 + DPAPI 包裹密钥                  | <a href="#">d59c4eb</a> |
| 审计工具结果接口参数顺序颠倒                                | 成功的工具调用被误判为失败，链路中断       | 修正调用方 + 调用点静态扫描回归测试                       | <a href="#">6577cdc</a> |

| 核心问题                   | 危害       | 修复                               | 提交      |
|------------------------|----------|----------------------------------|---------|
| 人名识别漏当代高频大姓、5 位以上裸金额漏检 | 核心实体召回不足 | 姓氏/角色触发词补全 + 金额正则扩展（评测复测召回 100%） | f8b766d |

表 3 安全内核级问题与修复（节选，均可在 git 历史追溯）

## 8 测试与质量保障

项目建立 5 个测试套件共 40+ 断言，覆盖隐私引擎、审计哈希链、记忆管理、工作流引擎与知识库管线，并新增隐私攻防评测（第 5 节）。全量回归通过。

| 测试套件                                | 覆盖范围                    | 结果      |
|-------------------------------------|-------------------------|---------|
| privacy_eval（攻防评测）                  | 黑盒/白盒/劫持/注入/无密钥 9 项实验   | ✅ 全部达标  |
| test_privacy_engine                 | 实体识别/脱敏/还原/nonce/strict | ✅ 通过    |
| test_audit_log                      | 哈希链/防篡改/调用点一致性扫描        | ✅ 14/14 |
| test_memory_optimize                | 摘要压缩/跨轮还原/序列化           | ✅ 6/6   |
| test_workflow / test_knowledge_base | 工作流/审批链/入库检索/向量库零明文     | ✅ 通过    |

表 4 测试套件汇总

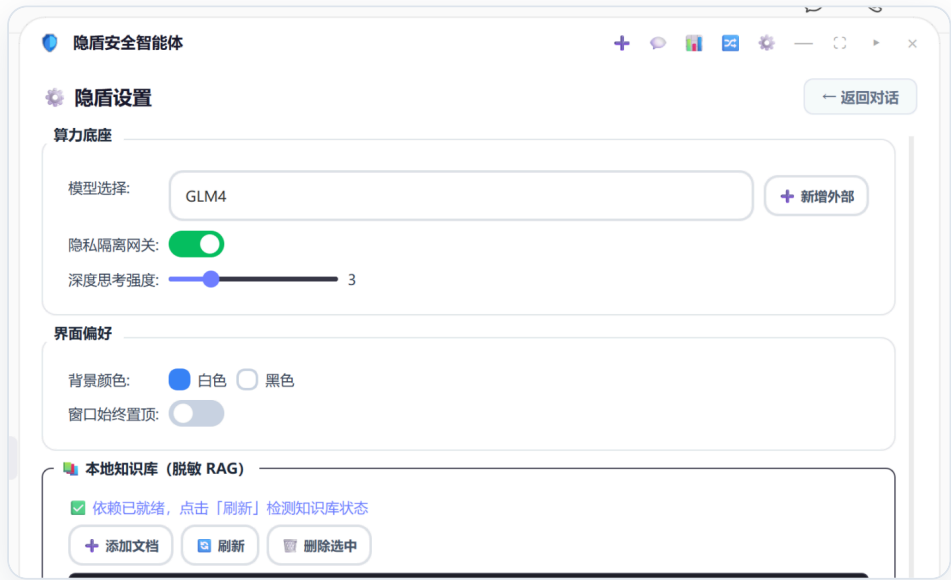


图 5 设置面板：模型选择、隐私隔离网关、深度思考强度与脱敏 RAG 知识库管理

## 9 结论与展望

隐盾把"让大模型安全地处理涉密办公数据"从使用约定下沉为架构约束：以内存级脱敏网关关闭出网与落盘泄露面，以沙箱/白名单/熔断审批约束物理副作用，以哈希链审计实现可追溯且自身不泄露，并以可复现的攻防实验量化了隐私防护效果（黑盒召回 100%、明文泄露 0、还原劫持 0、白盒持久化 0 命中）。核心问题均以 git 提交与回归测试留痕，形成"发现—修复—验证"的工程闭环。

### 下一步

- 引入 NER 模型补充规则式识别，提升开放域实体召回；
- 工具调用结果文件系统侧二次核验，杜绝小模型幻觉式"假成功"；
- 审计链头外部锚定，抵御整体重算哈希链；
- 在更大真实语料上扩展攻防评测，量化召回—误报权衡曲线，并引入差分隐私式度量作为补充。

本文档中的个人信息（张伟、13812345678 等）均为公开测试样例，不涉及任何真实个人数据。评测与截图取自真实运行环境（Windows 11 · RTX 3060 Ti · Ollama qwen2.5-7b / GLM-4-Flash 双算力）。